

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: batch, incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 2 - SQL, bancos, tabelas e Delta Lake

Databricks para Engenharia de Dados

I&D Capgemini Brasil, Setembro/2026





Objetivos do Dia 2

Ao final, você deve conseguir criar e consultar tabelas Delta com SQL no Databricks.

Entender SQL no Databricks

Por que SQL continua sendo essencial para Engenharia de Dados e como se conecta ao Lakehouse.

Diferenciar OLTP e OLAP

Separar a lógica de sistemas transacionais da lógica de plataformas analíticas.

Criar tabelas Delta

Usar comandos básicos para criar, carregar, consultar e alterar dados em tabelas Delta.

Executar laboratório

Carregar um dataset, registrar tabela e produzir consultas SQL básicas e intermediárias.

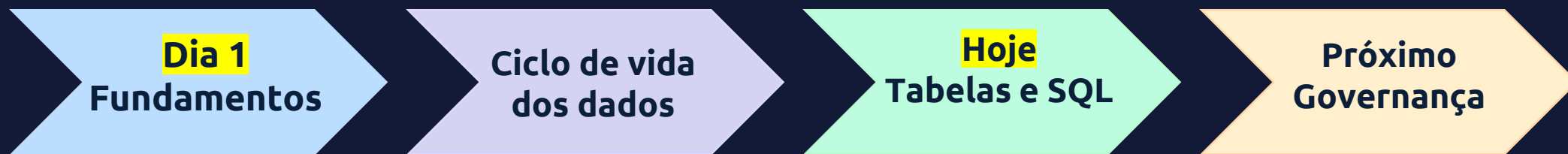
Construir base para Dia 3

Preparar entendimento para Unity Catalog, schemas, volumes e governança.



Conexão com o Dia 1

Do modelo mental para a manipulação prática dos dados.



Mensagem central

O Databricks não começa no código.

Começa no desenho correto do dado como ativo governado,
versionado e consultável.

Dia 2 - SQL, bancos, tabelas e Delta Lake

Bootcamp Databricks

I&D Capgemini Brasil



Por que SQL ainda é central?

SQL é a linguagem comum entre Engenharia, Analytics e consumo de dados.

Linguagem declarativa

O usuário descreve o resultado desejado, e o mecanismo de execução decide o plano físico.

Ponte com áreas de negócio

Analistas, engenheiros e times de BI conseguem compartilhar consultas, validações e regras.

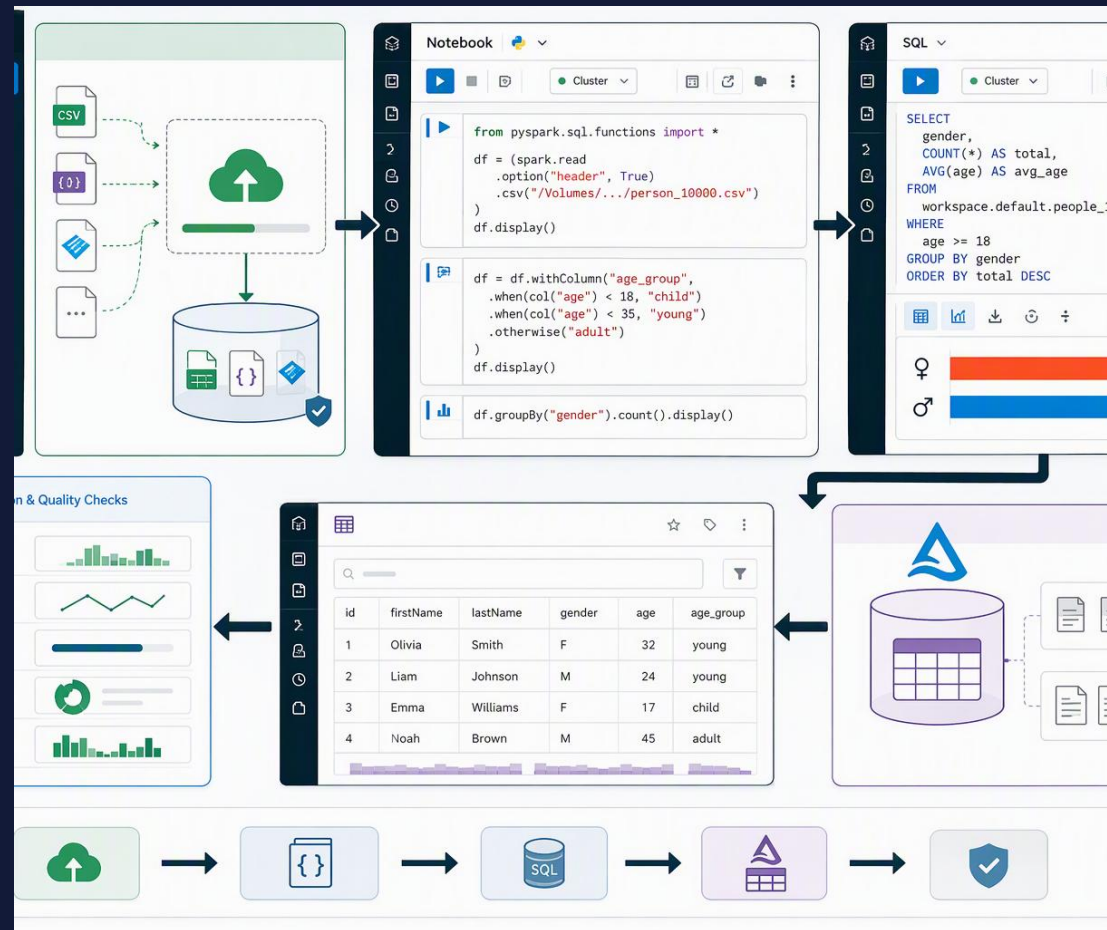
Base para pipelines

Muitas transformações Bronze/Silver/Gold podem ser expressas de forma simples e auditável em SQL.

SQL no Databricks não é “só consulta”:
é uma forma prática de criar, transformar, validar e publicar dados no Lakehouse.

OLTP vs OLAP

Antes de criar tabelas analíticas, é preciso entender a diferença de propósito.



OLTP

- Sistemas transacionais
- Registro de eventos de negócio
- Operações pequenas, frequentes e concorrentes

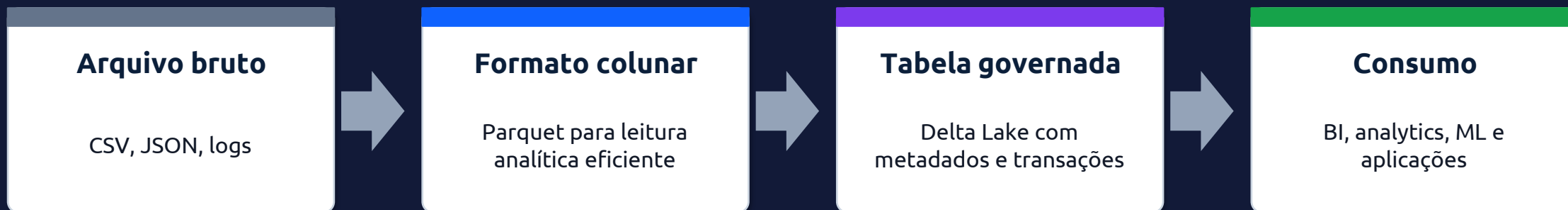
OLAP

- Plataformas analíticas
- Perguntas de negócio e BI
- Leituras grandes, agregações e histórico



Demonstração

O foco é organizar dados para leitura, histórico, reuso e escala.



No Lakehouse, a tabela passa a ser o contrato prático entre produção, governança e consumo.



Formatos de arquivo: o que muda na prática?

Nem todo formato oferece o mesmo nível de confiabilidade para workloads de dados.

Formato	Ponto forte	Limitação típica	Uso comum
CSV	Simples e universal	Sem tipagem robusta e sem transação	Entrada rápida e troca simples
JSON	Flexível para dados semiestruturados	Pode ser pesado e irregular	APIs e eventos
Parquet	Colunar e eficiente para analytics	Não resolve transações sozinho	Data lakes e consulta analítica
Delta	Parquet + log transacional	Requer suporte ao protocolo Delta	Tabelas confiáveis no Lakehouse

Para Engenharia de Dados em Databricks, Delta vira o padrão de trabalho para tabelas persistentes e confiáveis.

Delta Lake: por que importa?

Delta Lake adiciona confiabilidade transacional ao dado armazenado em formatos abertos.



ACID transactions

Operações de leitura e escrita consistentes para tabelas em workloads de dados.

Transaction log

Registro de alterações que permite controle de versões e histórico.

Batch + streaming

Foi desenvolvido com integração ao Structured Streaming para trabalhar com batch e streaming sobre uma cópia de dados.

Default no Databricks

No Databricks, tabelas são Delta Lake por padrão salvo quando especificado de outra forma.



Anatomia conceitual de uma tabela Delta

A tabela não é apenas um conjunto de arquivos.

Metadados e esquema

Contrato da tabela: nomes, tipos, comentários e propriedades

Log de transações

Histórico de operações que define versões consistentes

Arquivos Parquet

Dados físicos armazenados em cloud object storage

Tabela no catálogo

Objeto consultável por SQL, notebooks e BI

Mensagem didática: quando se cria uma tabela Delta, cria-se também um histórico operacional útil para confiabilidade, auditoria e evolução.



Tipos de tabela no Databricks

Precisa-se reconhecer os principais tipos antes de escolher onde salvar os dados.

Managed tables

Databricks gerencia metadados e arquivos de dados para novas tabelas que exigem performance otimizada.

External tables

Os dados ficam em armazenamento externo, enquanto o Unity Catalog gerencia metadados.

Temporary tables

Tabelas temporárias com escopo de sessão, úteis para dados intermediários.

Foreign tables

Acesso somente leitura a dados em sistemas externos via Lakehouse Federation.

Prática do curso:

começar com tabelas Delta gerenciadas para reduzir complexidade e focar na manipulação SQL.



Namespace: catalog.schema.table

O nome completo da tabela preserva contexto, governança e organização

catalog

Ambiente, domínio ou área de governança

schema

Agrupamento lógico dentro do catálogo

table

Objeto consultável e persistente

Exemplo

capgemini_academy.bronze.movies



Managed vs External: decisão prática

A diferença não é SQL; é gestão de dados e localização física.

Managed table

Databricks gerencia metadados e dados.
Ideal para novos datasets do treinamento e
para simplificar operação inicial.

External table

Os dados residem em local externo.
Útil quando a empresa já controla storage,
permissões e lifecycle fora da tabela.

Regra pedagógica do Dia 2:

reduzir variáveis e usar managed tables. Regra enterprise: decidir com governança, storage, segurança e ciclo de vida em mente.



SQL essencial: SELECT, FROM, WHERE

Comece pelo padrão que se usará em quase todo notebook SQL.

```
SELECT
  movieId, title, genres
FROM capgemini_academy.bronze.movies
WHERE genres LIKE '%Comedy%';
```

Leitura didática

SELECT escolhe colunas. FROM define a tabela. WHERE filtra linhas. O trio resolve a maior parte da exploração inicial.

Ponto de atenção

Sempre use nomes explícitos de tabela e evite depender de contexto implícito quando estiver ensinando boas práticas.

Datasets exemplo:
<https://grouplens.org/>



SQL essencial: agregações

Consulta analítica começa quando agrupamos e sumarizamos.

```
SELECT
  movieId,
  COUNT(*) AS total_avaliacoes,
  AVG(rating) AS avg_rating
FROM capgemini_academy.bronze.ratings
GROUP BY movieId
ORDER BY total_avaliacoes DESC;
```

Pergunta de negócio

Quantos registros existem por grupo? Qual a média?
Qual grupo é maior?

Conexão com Gold

Agregações aparecem de forma recorrente em data marts, visões de BI e métricas de negócio.

A agregação transforma registros transacionais de avaliação em uma visão analítica: quantidade de avaliações e média por filme.



SQL essencial: joins

Raramente uma resposta analítica vem de uma única tabela.

movies

ratings

movielf

Dica para laboratório:

O JOIN conecta a dimensão de filmes com os eventos de avaliação. Esse é o primeiro passo para sair de dados brutos e construir uma visão analítica reutilizável.



Criação de tabelas Delta

A criação de uma tabela pode partir de definição explícita ou de uma consulta.

CREATE TABLE

Define estrutura e registra objeto consultável no catálogo.

```
CREATE TABLE IF NOT EXISTS
capgemini_academy.sandbox.department (
  deptcode INT,
  deptname STRING,
  location STRING
);
```

```
INSERT INTO capgemini_academy.sandbox.
department
VALUES
  (10, 'FINANCE', 'EDINBURGH'),
  (20, 'SOFTWARE', 'PADDINGTON');
```

CREATE TABLE

Define estrutura e registra objeto consultável no catálogo.

CTAS

Cria tabela a partir do resultado de uma query, útil para materializar transformações.

Prática do Dia 2

Comece com uma tabela simples e deixe constraints, masks e row filters para módulos avançados.



Criação de tabelas Delta com CTAS

```
CREATE TABLE IF NOT EXISTS capgemini_academy.silver.movie_ratings_summary AS
SELECT
    m.movieId,
    m.title,
    m.genres,
    COUNT(r.rating) AS total_avaliacoes,
    AVG(r.rating) AS media_rating
FROM
    capgemini_academy.bronze.movies AS m
    JOIN capgemini_academy.bronze.ratings AS r
        ON m.movieId = r.movieId
GROUP BY m.movieId, m.title, m.genres;
```

CTAS materializa o resultado de uma consulta em uma nova tabela Delta. Aqui, promove-se dados da camada Bronze para uma primeira estrutura Silver analítica.



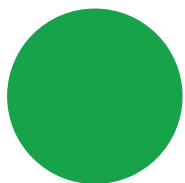
Operações básicas em tabelas Delta

Depois de criar a tabela, precisa-se praticar leitura e alterações controladas.



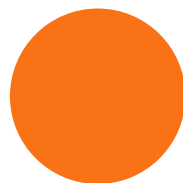
SELECT

Ler dados e validar
resultado



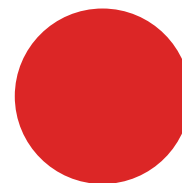
INSERT

Adicionar registros



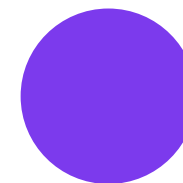
UPDATE

Corrigir valores existentes



DELETE

Remover registros quando
fizer sentido



MERGE

Atualizar e inserir em uma
única lógica

Cuidado didático:

UPDATE, DELETE e MERGE exigem conversa sobre impacto,
rastreabilidade e ambiente correto.



Schema enforcement e evolução

Confiabilidade começa por não aceitar qualquer dado de qualquer jeito.

Schema enforcement

Controla como o Databricks lida com mudanças de schema e tipos durante escritas.

Schema evolution

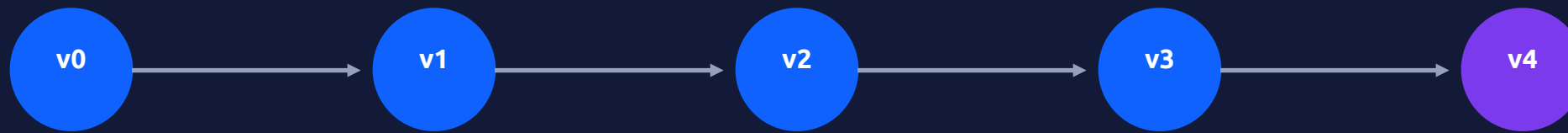
Permite evolução controlada quando novas colunas ou mudanças são esperadas.

Mensagem importante: tabela não é apenas armazenamento.
Tabela também é contrato de dados.



Histórico e time travel: conceito

Delta permite raciocinar sobre versões anteriores de uma tabela.



Ponto de cuidado:
políticas de retenção, VACUUM e governança
precisam ser tratados nos módulos de operação.

Boas práticas

Bootcamp Databricks

I&D Capgemini Brasil



Boas práticas

Aprenda a escrever SQL pensando em operação desde o início.

- Usar nomes qualificados sempre que possível: catalog.schema.table.
- Separar exploração, criação de tabela e validação em células diferentes.
- Evitar sobrescrever dados sem entender impacto e ambiente.
- Validar volume, schema e amostras antes de considerar a carga concluída.
- Registrar comentários simples no notebook para explicar intenção da query.

A maturidade começa nos pequenos hábitos: nome, contexto, validação e rastreabilidade.



Demonstração guiada

1

Abrir notebook SQL/Python

Conectar em compute disponível

2

Criar ou selecionar catálogo/schema

Usar namespace do treinamento

3

Ler CSV ou Parquet

Mostrar display e inferência de schema

4

Criar tabela Delta

Registrar tabela para consulta SQL

5

Executar SELECT, GROUP BY e filtro

Validar resultado e discutir intenção

6

Criar tabela Silver com CTAS

Apenas se o ambiente estiver preparado

Atividades práticas

Bootcamp Databricks

I&D Capgemini Brasil



Laboratório: Parte 1 — preparar dados

```
-- 1. Definir contexto
USE CATALOG capgemini_academy;
USE SCHEMA bronze;

-- 2. Validar tabelas existentes
SHOW TABLES;

-- 3. Se dataset estiver em volume, validar arquivos
-- Exemplo conceitual: /Volumes/capgemini_academy
/landing/files/movielens/movies.csv
```



Laboratório: Parte 2 — criar tabela Delta

```
-- Opção A: criar tabela por SQL
CREATE TABLE IF NOT EXISTS department (
    deptcode INT,
    deptname STRING,
    location STRING
);

INSERT INTO department VALUES
    (10, 'FINANCE', 'EDINBURGH'),
    (20, 'SOFTWARE', 'PADDINGTON');

SELECT * FROM department;
```



Laboratório: Parte 3 — consultas e validação

```
SELECT COUNT(*) AS total  
FROM department;
```

```
SELECT location, COUNT(*) AS qtd  
FROM department  
GROUP BY location  
ORDER BY qtd DESC;
```

```
DESCRIBE TABLE department;
```

Validação mínima

A tabela existe, tem schema, tem registros e retorna consultas sem erro.

Entregável do aluno

Notebook com comandos executados, resultados e breve explicação do que cada consulta valida.

Objetivo: Reutilizar as tabelas Bronze criadas no Dia 1 para praticar SQL, validar dados, executar filtros, agregações e joins, e materializar a primeira tabela analítica nas camadas Silver e Gold.

Narrativa: no Dia 1 criamos a fundação Lakehouse. No Dia 2 transformaremos dados carregados em consultas, validações e primeiras tabelas analíticas.

Jornada prática do laboratório

Fluxo recomendado para conduzir o exercício no SQL Editor ou notebook SQL.

1

Contexto SQL

CTX

Selecionar capgemini_academy e schema bronze

2

Validar Bronze

BR

Confirmar a existência de movies e ratings criadas no Dia 1.

3

Explorar movies

SQL

Aplicar SELECT, LIMIT e WHERE com filtros de gênero.

4

Explorar ratings

EVT

Contar registros e entender avaliações como eventos.

5

Agregações

AGG

Usar GROUP BY, COUNT e AVG para gerar métricas por filme.

6

JOIN

JOIN

Conectar movies + ratings pelo movieid para criar visão analítica.

7

Silver CTAS

SLV

Criar silver.movie_ratings_summary com CREATE TABLE AS SELECT.

8

Gold desafio

GLD

Criar view gold.vw_topRated_movies considerando apenas films com pelo menos 50 avaliações.

Entregável final

capgemini_academy



Checklist do laboratório

- ✓ Contexto SQL definido
- ✓ Bronze validada
- ✓ SELECT e WHERE executados
- ✓ GROUP BY e AVG aplicados
- ✓ JOIN movies + ratings
- ✓ Silver e Gold criadas

Comandos centrais

```
USE CATALOG capgemini_academy;
SHOW TABLES IN capgemini_academy.bronze;
SELECT rating, COUNT(*) total FROM bronze.ratings
GROUP BY rating ORDER BY rating;
CREATE OR REPLACE TABLE silver.movie_ratings_summary AS
SELECT m.title, COUNT(*) total, AVG(r.rating) media
```

Conhecimento se adquire estudando. Confiança se conquista praticando.



Quiz de fixação

- 1) Qual a principal diferença entre OLTP e OLAP?
- 2) Por que Delta Lake é mais adequado que arquivos Parquet soltos para tabelas de engenharia?
- 3) O que representa `catalog.schema.table`?
- 4) Quando você escolheria uma tabela `managed` no treinamento?
- 5) Quais validações mínimas são esperadas após criar uma tabela?

Sugestão: respostas curtas e conectada ao laboratório executado.



Fechamento do Dia 2

SQL é base operacional

A mesma linguagem serve para explorar, validar, transformar e documentar dados.

Delta Lake é confiabilidade

Delta adiciona transaction log, suporte a transações e histórico para tabelas no Lakehouse.

Tabela é contrato

Nome, schema, tipo, histórico e localização precisam ser intencionais.

Próximo encontro

Unity Catalog, schemas, volumes e governança



Desafio do Dia 2 – Assistir e resumir

- Como eu aprenderia SQL RAPIDAMENTE
<https://www.youtube.com/watch?v=kkKPD7khA54>
- Fundamentos do Delta Lake
<https://www.youtube.com/watch?v=W1wwDTj2NJE>

Thank you.

Make it real.

Capgemini 